

Module: - Frontend – CSS and CSS3

JavaScript Introduction

Question 1: What is JavaScript? Explain the role of JavaScript in web development.

JavaScript (JS) is a high-level, interpreted programming language primarily used to create interactive and dynamic content on the web. It is one of the three core technologies of the World Wide Web, alongside HTML (structure) and CSS (styling).

While HTML provides the bones and CSS provides the skin, JavaScript provides the behavior that makes a website "come alive".

Here is some role of JavaScript in web development:

- **Client-Side Interactivity:** It runs directly in the user's browser, allowing for immediate responses to actions like button clicks, mouse movements, or form submissions without reloading the entire page.
- **Dynamic Content Updates:** JavaScript can modify HTML and CSS in real-time through the Document Object Model (DOM). This enables features like image carousels, live countdowns, and refreshing social media feeds.
- **Asynchronous Programming:** Using techniques like AJAX, Promises, and `async/await`, JavaScript can fetch data from a server in the background, allowing pages to remain responsive while loading new information.
- **Form Validation:** It checks user input (e.g., verifying a valid email format) instantly on the client side before any data is sent to the server, improving the user experience and reducing server load.
- **Server-Side Development:** With environments like Node.js, developers can use JavaScript to build back-end logic, manage databases, and create APIs.
- **Frameworks & Libraries:** Modern development relies heavily on a massive ecosystem, including React (UI components), Angular (full-featured apps), and Vue.js (versatile interfaces).
- **Cross-Platform Apps:** Frameworks like React Native and Electron allow developers to build mobile and desktop applications using their existing JavaScript skills.

Question 2: How is JavaScript different from other programming languages like Python or Java?

Table 1 Difference between JavaScript, Python and Java

Feature	JavaScript	Python	Java
Type	Interpreted	Interpreted	Compiled
Main Use	Web (Frontend + Backend)	Data Science, AI, Backend	Enterprise apps, Android
Syntax	Flexible	Very simple & readable	Strict & verbose
Execution	Browser + Server (Node.js)	Server	JVM
Typing	Dynamic & Weak	Dynamic & Strong	Static & Strong

Speed	Fast (V8 engine)	Moderate	Very fast
Concurrency	Event-driven (Single-threaded)	Process-based (GIL)	Multi-threaded

1. Core Philosophy and Execution

JavaScript was designed for browser-side interactivity. It is mostly interpreted in real-time by engines (like Chrome's V8), using Just-In-Time (JIT) compilation to run scripts instantly without a manual compile step.

Python focuses on human readability and "batteries-included" simplicity. It is also interpreted but generally slower for real-time web tasks compared to JavaScript's highly optimized browser engines.

Java follows a "Write Once, Run Anywhere" (WORA) philosophy. It is compiled into bytecode before running on a Java Virtual Machine (JVM), making it more robust and faster for complex, large-scale systems.

2. Type Systems and Safety

JavaScript is weakly typed. It allows implicit type conversions (e.g., adding a number to a string), which can lead to unpredictable "silent" errors during runtime.

Python is strongly typed. While you don't declare types upfront, the language will throw an error if you try to perform incompatible operations, making it easier to debug than JavaScript.

Java is statically typed. You must declare variable types (e.g., `int x = 5;`) at the start. The compiler catches most errors before the code even runs, providing the highest level of safety among the three.

3. Object-Oriented Models

JavaScript uses Prototypal Inheritance. Objects inherit directly from other objects rather than from fixed "blueprints". This is highly flexible but often confusing for those used to traditional programming.

Python and Java use Classical Inheritance, where classes serve as blueprints for objects. This structure is more rigid and standardized across the software industry.

4. Handling Multiple Tasks

JavaScript is single-threaded and event-driven. It uses an "event loop" to handle many simultaneous connections (like in Node.js) without waiting for each one to finish, making it excellent for real-time apps like chat or live feeds.

Java supports multi-threading. It can use multiple CPU cores simultaneously to perform different heavy calculations at once, which is why it's preferred for massive data processing and banking systems.

Question 3: Discuss the use of `<script>` tag in HTML. How can you link an external JavaScript file to an HTML document?

The `<script>` tag is the standard HTML element used to embed or link executable client-side scripts, primarily JavaScript, into a web document. It enables developers to add interactivity, such as form validation, image manipulation, and dynamic content updates, to otherwise static web pages.

1. Inline JavaScript

```
<button onclick="alert('Hello')">Click Me</button>
```

2. Internal JavaScript

```
<script>
    console.log("Hello World");
</script>
```

3. External JavaScript (Best Practice)

Step 1: Create a JS file

script.js

```
console.log("External JS loaded");
```

Step 2: Link it in HTML

```
<script src="script.js"></script>
```

Place before closing </body>:

```
<body>
```

```
  <!-- content -->
```

```
  <script src="script.js"></script>
```

```
</body>
```

Variables and Data Types

Question 1: What are variables in JavaScript? How do you declare a variable using var, let, and const?

In JavaScript, variables are named containers used for storing and managing data values that can be reused or modified throughout a program.

You declare variables using three keywords: var, let, and const.

1. Using var

Syntax: `var name = "value";`

Scope: It is function-scoped or globally scoped.

Key Traits: You can re-declare and update var variables within the same scope. They are hoisted and initialized with undefined, meaning you can access them before the actual declaration without a fatal error.

2. Using let

Introduced in ES6 (2015), let is the modern standard for variables that might change.

Syntax: `let age = 25;`

Scope: It is block-scoped, meaning it only exists within the curly braces {} where it is defined.

Key Traits: You can update (reassign) the value, but you cannot re-declare it in the same scope. It is hoisted but stays in a "temporal dead zone" until initialized, so accessing it before declaration causes a ReferenceError.

3. Using const

Also introduced in ES6, const is for values that should remain constant.

Syntax: `const PI = 3.14;`

Scope: Like let, it is block-scoped.

Key Traits: It cannot be reassigned or re-declared. You must initialize it with a value at the time of declaration. While the reference itself is constant, the properties of an object or elements in an array declared with const can still be changed.

Table 2 Differende between var, let and const

Feature	var	let	const
Scope	Function / Global	Block	Block
Reassignable	Yes	Yes	No
Redeclarable	Yes	No	No

Question 2: Explain the different data types in JavaScript. Provide examples for each.

In JavaScript, data types are broadly divided into two categories: Primitive and Non-Primitive (Reference) types. JavaScript is dynamically typed, meaning variables are not bound to a specific type and can change during execution.

1. Primitive Data Types

Primitive types represent single, immutable values stored directly in memory. There are seven basic primitive types:

String: Textual data enclosed in single ('), double ("), or backticks (`).

Example: `let name = "Alice";`

Number: Represents both integers and floating-point numbers.

Example: `let age = 30;` or `let pi = 3.14;`

BigInt: Used for integers larger than the safe limit for the Number type ($2^{53} - 1$).

Example: `let largeNum = 9007199254740991n;`

Boolean: Represents logical entities with only two values: true or false.

Example: `let isVisible = true;`

Undefined: A variable that has been declared but not yet assigned a value.

Example: `let x;` // x is undefined

Null: Represents the intentional absence of any object value.

Example: `let user = null;`

Symbol: Represents a unique and immutable identifier, often used as object property keys.

Example: `let id = Symbol("id");`

2. Non-Primitive (Reference) Data Types

Non-primitive types are complex data structures that store collections of values and are accessed by reference.

Object: A collection of key-value pairs where keys are strings or symbols and values can be any type.

Example: `let person = { name: "Bob", age: 25 };`

Array: A specialized type of object used to store ordered lists of data.

Example: `let colors = ["red", "green", "blue"];`

Function: Reusable blocks of code that can be assigned to variables and passed around as objects.

Example: `function greet() { return "Hello!"; }`

Question 3: What is the difference between undefined and null in JavaScript?

In JavaScript, both null and undefined represent the absence of a value, but they differ in how they are triggered and their technical behavior.

Core Difference:

- **undefined** is automatically assigned by JavaScript when a variable has been declared but not yet given a value.
- **null** is an assignment value that you, the developer, manually assign to indicate that a variable is intentionally empty.

Table 3 Comparison between undefined and null

Feature	undefined	null
Meaning	Absence of a defined value	Intentional absence of any value
Assignment	Implicit (done by JS engine)	Explicit (done by developer)
Type (typeof)	"undefined"	"object" (historical quirk)
Math Operations	Converts to NaN	Converts to 0
JSON	Omitted from serialization	Preserved as null

Common Scenarios for undefined:

- Uninitialized Variables: `let x;` results in `x` being undefined.
- Missing Function Arguments: If a function expects a parameter but you don't pass it, that parameter becomes undefined inside the function.
- Missing Returns: A function that doesn't explicitly return a value will return undefined by default.
- Non-existent Properties: Trying to access a property that doesn't exist on an object returns undefined.

JavaScript Operators

Question 1: What are the different types of operators in JavaScript? Explain with examples.

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators

In JavaScript, operators are symbols used to perform operations on variables and values. They are essential for manipulating data and controlling program flow.

The four primary types of operators you requested are explained below:

1. Arithmetic Operators:

These operators are used to perform mathematical calculations between numeric values.

- **Addition (+):** Adds two values. (e.g., $5 + 10$ results in 15).
- **Subtraction (-):** Subtracts the right value from the left. (e.g., $10 - 5$ results in 5).
- **Multiplication (*):** Multiplies two values. (e.g., $5 * 10$ results in 50).
- **Division (/):** Divides the left operand by the right. (e.g., $10 / 5$ results in 2).
- **Modulus (%):** Returns the remainder after division. (e.g., $10 \% 3$ results in 1).
- **Exponentiation (**):** Raises the first value to the power of the second. (e.g., $4 ** 2$ results in 16).
- **Increment (++) & Decrement (--):** Increases or decreases a value by 1.

2. Assignment Operators:

Assignment operators are used to store or update values in variables.

- **Simple Assignment (=):** Assigns the right-side value to the left-side variable. (e.g., $x = 5$).
- **Addition Assignment (+=):** Adds a value to the current variable and assigns the result back. (e.g., $x += 5$ is $x = x + 5$).
- **Subtraction Assignment (-=):** Subtracts a value and assigns the result. (e.g., $x -= 3$ is $x = x - 3$).
- **Multiplication/Division Assignment (*=, /=):** Multiplies or divides the current value and assigns the result back.

3. Comparison Operators:

These operators compare two values and return a boolean result: true or false.

- **Equal To (==):** Returns true if values are equal (allows type conversion). (e.g., $5 == "5"$ is true).
- **Strict Equal (===):** Returns true only if both value and type are identical. (e.g., $5 === "5"$ is false).
- **Greater Than (>) & Less Than (<):** Checks if one value is larger or smaller than another.
- **Greater/Less Than or Equal To (>=, <=):** Checks if a value is at least or at most equal to another.

- **Not Equal (!=) & Strict Not Equal (!==):** Checks for inequality, either with or without type conversion.

4. Logical Operators

Logical operators combine multiple conditions or invert boolean values.

- **Logical AND (&&):** Returns true only if all conditions are true. (e.g., `(5 > 3) && (2 < 4)` is true).
- **Logical OR (||):** Returns true if at least one condition is true. (e.g., `(5 > 3) || (2 > 4)` is true).
- **Logical NOT (!):** Reverses the boolean value. (e.g., `!true` becomes false).

Question 2: What is the difference between == and === in JavaScript?

The primary difference between `==` and `===` in JavaScript is that `==` performs type conversion (coercion) before comparing values, while `===` does not and requires both the value and the type to be identical.

Table 4 Comparison of loose equality and strict equality

Feature	<code>==</code> (Loose Equality)	<code>===</code> (Strict Equality)
Name	Abstract/Loose Equality	Strict Equality/Identity
Type Coercion	Yes, converts types before checking	No, does not convert types
Criteria	Returns true if values match after conversion	Returns true only if value and type match
Usage	Rare; mostly used for checking null/undefined	Recommended default for predictable code

Number vs. String:

`5 == "5"` is true because the string "5" is converted to the number 5.

`5 === "5"` is false because they are different types (Number vs. String).

Boolean vs. Number:

`true == 1` is true because true is coerced to the number 1.

`true === 1` is false because their types do not match.

Null and Undefined:

`null == undefined` is true because they are considered loosely equal.

`null === undefined` is false because they are distinct types.

Control Flow (If-Else, Switch)

Question 1: What is control flow in JavaScript? Explain how if-else statements work with an example.

In JavaScript, control flow refers to the order in which the interpreter executes statements. By default, code runs sequentially from top to bottom. Control flow structures allow you to break this linear path, enabling your program to make decisions, repeat tasks, or skip sections of code based on specific conditions.

The if-else statement is a fundamental control flow tool used for decision-making. It evaluates a condition and branches the execution path based on whether that condition is true or false.

- **if block:** This section only runs if the specified condition evaluates to a truthy value (something that behaves like true).
- **else block:** This optional section only runs if the condition is falsy (something that behaves like false).

Example: Checking a User's Age

In this scenario, the program decides which message to display based on the value of a variable.

```
let age = 17;
if (age >= 18) {
  // This code runs only if age is 18 or greater
  console.log("You are an adult.");
} else {
  // This code runs only if the condition above is false
  console.log("You are a minor.");
}
// Output: "You are a minor."
```

Question 2: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

A switch statement in JavaScript is a control-flow tool that evaluates a single expression once and compares its result against various "cases" using strict equality (===). It is often used as a cleaner, more organized alternative to long chains of if...else if statements.

How Switch Statements Work:

The execution follows a specific sequence:

- **Evaluation:** The main expression is evaluated just once at the start.
- **Comparison:** The result is compared against each case value from top to bottom.
- **Execution:** When a match is found, the code block for that case runs.
- **Break:** A break keyword is typically used at the end of each case to exit the switch block. Without it, execution "falls through" to the next case automatically, regardless of whether that case matches.
- **Default Case:** An optional default case acts as a fallback, executing only if no other cases match.

When to Use Switch vs. If-Else:

The choice often depends on the complexity of the conditions and the number of values being checked.

Table 5 When to use Switch vs If-Else

Feature	Use Switch When...	Use If-Else When...
Condition Type	You are checking a single variable against multiple discrete values.	You have complex logic, ranges (e.g., score > 90), or multiple different variables.
Number of Cases	There are many conditions (typically 3 or more) to improve readability	There are only 1 or 2 simple conditions.
Comparison	You only need to check for exact equality (==).	You need logical operators (like && or `
Performance	Performance is slightly better for large sets because engines can use "jump tables" to find the match directly.	Performance is standard for a few checks but can slow down with long sequential chains.

Readability: Switch is generally easier to read for "menu-like" selections where one variable has many potential states.

Flexibility: If-else is far more flexible, as it can evaluate any expression that results in a boolean value.

Maintenance: Adding or removing cases in a switch is often simpler and less prone to errors than restructuring deeply nested if statements.

Loops (For, While, Do-While)

Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

JavaScript uses three primary types of loops to execute blocks of code repeatedly until a specific condition is met.

1. The for Loop

The for loop is best used when you know exactly how many times you want the code to run. It combines initialization, condition checking, and incrementing into a single line.

Example:

```
// Prints numbers 0 through 4
for (let i = 0; i < 5; i++) {
  console.log("Iteration:", i);
}
```

2. The while Loop

The while loop repeats a block of code as long as a specified condition remains true. It is an "entry-controlled" loop, meaning it checks the condition before executing the code.

Example:

```
let count = 0;
while (count < 3) {
  console.log("Count is:", count);
  count++; // Important to increment to avoid an infinite loop
}
```

3. The do-while Loop

The do-while loop is similar to the while loop, but with one critical difference: it is an "exit-controlled" loop. It executes the code block first and then checks the condition, guaranteeing that the code runs at least once even if the condition is initially false.

Example:

```
let x = 10;
do {
  console.log("This will run exactly once.");
  x++;
} while (x < 5); // Condition is false, so it stops after one run
```

Question 2: What is the difference between a while loop and a do-while loop?

The primary difference is when the condition is checked. A while loop checks the condition before executing the code block, whereas a do-while loop executes the code block first and checks the condition afterward.

Table 6 Difference between while loop and do-while loop

Feature	while Loop	do-while Loop
Condition Check	At the beginning (Entry-controlled)	At the end (Exit-controlled)
Minimum Executions	0 times; it may never run if the condition is initially false	1 time; it is guaranteed to execute at least once regardless of the condition
Use Case	When you only want code to run if a condition is already met	When you need the code to run at least once, such as for input validation or displaying a menu
Syntax (C-style)	<code>while(condition) { ... }</code>	<code>do { ... } while(condition);</code>
Semicolon	No semicolon after the condition	Required after the condition

Functions

Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.

In JavaScript, functions are **reusable blocks of code** designed to perform specific tasks. They allow you to write a set of instructions once and execute them multiple times with different inputs, which helps in organizing and modularizing your code. Unlike many other languages, JavaScript functions are first-class objects, meaning they can be stored in variables, passed as arguments to other functions, and even returned from other functions.

To create a standard function (known as a function declaration), you use the following syntax:

- **function keyword:** Identifies the start of the declaration.
- **Function Name:** A unique identifier used to refer to the function later.
- **Parameters:** Optional placeholders enclosed in parentheses () that accept input values (arguments) when the function is called.
- **Function Body:** The code to be executed, enclosed in curly braces {}.
- **Example:**

```
function greet(name) {  
    return "Hello, " + name + "!";  
}
```

Defining a function does not execute it; it simply saves the instructions for later. To execute the code within a function, you must call (or invoke) it by using its name followed by parentheses ().

If the function requires parameters, you pass the actual values (arguments) inside the parentheses.

The function will execute its logic and optionally return a value to the caller using the return statement.

Example:

```
let message = greet("Alice"); // Calling the function with "Alice" as the argument  
console.log(message); // Output: "Hello, Alice!"
```

Beyond standard declarations, JavaScript offers several modern alternatives:

- **Function Expressions:** Storing a function (often anonymous) inside a variable.
- **Arrow Functions:** A concise syntax introduced in ES6 using the => operator.
- **Immediately Invoked Function Expressions (IIFEs):** Functions that run automatically as soon as they are defined.

Question 2: What is the difference between a function declaration and a function expression?

The primary difference between a function declaration and a function expression is when the function is created and made available for use by the JavaScript engine.

Table 7 Comparison between Function Declaration and Function Expression

Feature	Function Declaration	Function Expression
Syntax	Defined as a standalone statement.	Defined as part of an expression, often assigned to a variable.
Hoisting	Fully hoisted; can be called before it is defined in the code.	Not hoisted; can only be called after the interpreter reaches the definition.
Naming	Must have a name.	Can be anonymous or named.
Execution	Processed before any other code is executed.	Executed only when the interpreter reaches that specific line.

1. Function Declaration:

A function declaration is a separate statement that declares a named function.

Hoisting: JavaScript "hoists" these declarations to the top of their scope during the initialization stage. This allows you to call the function even if it is written later in the script.

Use Case: Best for general-purpose, reusable functions that you want available globally or throughout a specific scope.

Example:

```
greet(); // Works due to hoisting
function greet() {
  console.log("Hello!");
}
```

2. Function Expression

A function expression creates a function as a value within another syntax construct, typically by assigning it to a variable.

Availability: These are only created when the execution flow actually reaches them. If you try to call one before its definition, you will get an error.

Use Case: Ideal for callbacks (functions passed as arguments), Immediately Invoked Function Expressions (IIFEs), and dynamic function assignments.

Example:

```
sayHi(); // Error: Cannot access 'sayHi' before initialization
const sayHi = function() {
  console.log("Hi!");
};
```

When to Use Which?

In most cases, The Modern JavaScript Tutorial recommends using function declarations first because they provide more flexibility in code organization and are generally more readable. Use function expressions specifically when you need the function to be a value or when you want to limit its scope to prevent polluting the global namespace.

Question 3: Discuss the concept of parameters and return values in functions.

In computer programming, functions act as self-contained blocks of code designed to perform specific tasks. The concepts of parameters and return values are the primary mechanisms for moving data into and out of these blocks.

Parameters: Sending Data In

Parameters are variables specified in a function's definition that act as placeholders for the data it needs to do its job.

Parameters vs. Arguments: A parameter is the variable name listed in the function definition (e.g., x and y), while an argument is the actual value passed to the function when it is called (e.g., 5 and 10).

Types of Parameters:

- **Positional:** The most common type, where the order of arguments passed must match the order in the definition.
- **Keyword/Named:** Arguments are passed by explicitly naming the parameter, allowing them to be in any order.
- **Default:** Parameters that use a preset value if the caller doesn't provide one.
- **Variadic:** Allows a function to accept an arbitrary number of arguments, often handled as an array or list inside the function.

Return Values: Getting Results Out

A return value is the output a function sends back to the part of the program that called it once its task is complete.

The return Statement: This keyword terminates the function's execution immediately and passes a specific value back to the caller.

Communication: Unlike print(), which just displays text on a screen, a return value allows the program to store the result in a variable or use it in further calculations.

Return Types: Every function typically has a return type (e.g., int, string, bool). If a function performs an action but doesn't need to send data back, its return type is often called void or None.

Multiple Returns: While most languages traditionally return one value, some allow returning multiple values using structures like tuples.

Using parameters and return values allows for modular and reusable code. Instead of writing a new block of logic for every specific scenario, you can write one generic function that takes different inputs (parameters) and provides specific results (return values) for any part of your program to use.

Arrays

Question 1: What is an array in JavaScript? How do you declare and initialize an array?

In JavaScript, an array is an ordered list of values designed for storing data collections under a single variable name. Although they are technically a special type of global object, arrays are optimized to handle lists of data and include built-in properties (like `length`) and methods (like `push` or `pop`) for easy manipulation.

Key Characteristics of array

- **Zero-indexed:** The first item is at index 0, the second at 1, and so on.
- **Dynamic size:** They can grow or shrink automatically as you add or remove items.
- **Heterogeneous:** A single array can store different data types at once, such as numbers, strings, objects, and even other arrays.

There are two primary ways to create an array in JavaScript.

1. Array Literal Notation (Preferred):

This is the simplest and most common method. You use square brackets `[]` and separate elements with commas.

```
// Declare an empty array
const emptyArray = [];

// Declare and initialize with values
const fruits = ["Apple", "Banana", "Cherry"];
```

2. Array Constructor Syntax:

This method uses the `new Array()` constructor. It is generally less common but still valid.

```
// Declare and initialize with values
const numbers = new Array(10, 20, 30);

// Declare an array with a specific length (pre-allocated empty slots)
const preSizedArray = new Array(5); // Creates an array of length 5 with undefined slots
```

It is a best practice to use Array Literal Notation (`[]`) because it is shorter, more readable, and faster to execute. Additionally, the constructor has a confusing behavior: passing a single number like `new Array(5)` creates an array with 5 empty slots, whereas `[5]` creates an array containing only the number 5.

Question 2: Explain the methods `push()`, `pop()`, `shift()`, and `unshift()` used in arrays.

In JavaScript, `push()`, `pop()`, `shift()`, and `unshift()` are fundamental methods for adding or removing elements from an array. All four are mutating methods, meaning they directly modify the original array rather than creating a new one.

Adding and Removing from the End

These methods are highly efficient ($O(1)$ time complexity) because they don't require re-indexing the existing elements.

push(): Adds one or more elements to the end of the array.

Returns: The new length of the array.

pop(): Removes the last element from the array.

Returns: The removed element (or undefined if the array was empty).

Adding and Removing from the Beginning

These methods are generally slower ($O(n)$ time complexity) for large arrays because they force every remaining element to be moved to a new index.

unshift(): Adds one or more elements to the beginning of the array.

Returns: The new length of the array.

shift(): Removes the first element from the array.

Returns: The removed element (or undefined if the array was empty).

Table 8 Method used in Array

Method	Action	Location	Returns	Efficiency
push()	Add	End	New length	Fast ($O(1)$)
pop()	Remove	End	Removed item	Fast ($O(1)$)
unshift()	Add	Start	New length	Slower ($O(n)$)
shift()	Remove	Start	Removed item	Slower ($O(n)$)

A common way to remember these is that the longer words (push, unshift) add elements and make the array longer, while the shorter words (pop, shift) remove elements and make the array shorter.

Objects

Question 1: What is an object in JavaScript? How are objects different from arrays?

An object in JavaScript is a mutable data type that stores collections of data as key-value pairs, where each "key" (or property name) is a string or symbol that maps to a "value" of any data type. While arrays are actually a specialized type of object in JavaScript, they are functionally different in how they organize and access data.

Table 9 Difference between object and array

Feature	JavaScript Objects	JavaScript Arrays
Structure	Unordered collection of key-value pairs	Ordered list of values
Indexing	Named keys (e.g., person.name)	Numeric indexes starting at 0 (e.g., list[0])
Access Methods	Dot notation (obj.key) or bracket notation (obj['key'])	Bracket notation with numeric index (arr[0])
Length Property	No built-in lengthproperty	Automatically updated lengthproperty
Syntax	Curly braces { }	Square brackets []
Iteration	for...in loop or Object.keys()	for, for...of, or forEach()

- Use Objects when you are describing a single entity with specific characteristics (e.g., a "User" with a name, email, and age) or when the relationship between a named key and its value is vital.
- Use Arrays when you need an ordered list of similar items (e.g., a list of usernames) or when you need to maintain the specific sequence of data.

Question 2: Explain how to access and update object properties using dot notation and bracket notation.

In JavaScript, you can access and update object properties using dot notation or bracket notation. Dot notation is generally preferred for its readability, while bracket notation is essential for handling dynamic keys or property names that are not valid identifiers.

1. Dot Notation

Dot notation uses a period (.) followed by the property name.

Accessing: object.property

Updating: object.property = newValue

Rules: It only works with valid JavaScript identifiers (no spaces, hyphens, or starting with numbers).

Example:

```
const user = { name: "Alice", age: 25 };
```

```
// Access
    console.log(user.name); // "Alice"

// Update
    user.age = 26;
    console.log(user.age); // 26
```

2. Bracket Notation

Bracket notation uses square brackets [] containing the property name as a string or an expression that evaluates to a string.

Accessing: object["property"]

Updating: object["property"] = newValue

Flexibility: It can handle any string, including those with spaces, special characters, or numbers.

Example:

```
const car = { "brand-name": "Toyota", year: 2020 };
// Access (Required for names with hyphens)
    console.log(car["brand-name"]); // "Toyota"

// Update
    car["year"] = 2021;
```

Table 10 Comparison of Dot Notation and Bracket Notation

Feature	Dot Notation	Bracket Notation
Readability	Clean and concise	More verbose
Dynamic Keys	No: Cannot use variables	Yes: Essential for variables
Special Characters	No: Limited to identifiers	Yes: Handles spaces/hyphens

When to use Bracket Notation:

Using Variables: When the property name is stored in a variable.

```
const key = "name";
console.log(user[key]); // Returns user["name"]
```

Invalid Identifiers: For keys that include spaces or start with numbers (e.g., obj["2nd place"]).

Dynamic Expressions: When the property name is computed at runtime.

JavaScript Events

Question 1: What are JavaScript events? Explain the role of event listeners.

In JavaScript, events are signals from the browser that something significant has happened, while event listeners are the mechanism used to detect and respond to those signals. Together, they transform static web pages into interactive applications.

Events are "actions" or "occurrences" that take place in the browser environment. They can be triggered by a user (like clicking a button) or by the system itself (like a page finishing loading).

Common types of events include:

- **Mouse Events:** click, mouseover (hovering), dblclick.
- **Keyboard Events:** keydown (pressing a key), keyup (releasing it).
- **Form Events:** submit (sending a form), change (input value updates).
- **Window Events:** load (page finished loading), resize, scroll.

The Role of Event Listeners:

An event listener is a procedure that "waits" for a specific event to occur on a particular element. Its primary roles include:

- **Detection:** It continuously "watches" an element (like a button) for a specific trigger (like a click) without requiring the code to constantly check manually.
- **Execution:** When the event occurs, the listener automatically triggers a function, often called an event handler, to perform a task—such as showing an alert or updating a database.
- **Separation of Concerns:** Using the recommended `addEventListener()` method allows you to keep JavaScript logic separate from HTML markup, leading to cleaner and more maintainable code.
- **Multiple Responses:** Unlike older methods (like `onclick` attributes), event listeners allow you to attach multiple different functions to the same event on a single element.

Basic Syntax:

To set up a listener, you specify the event type and the callback function to run:

```
const btn = document.querySelector('button');  
// Role: Listen for a 'click' and then run the function  
btn.addEventListener('click', function() {  
    alert('Button was clicked!');  
});
```

When an event is triggered, the browser automatically creates an Event Object. This object is passed to the listener and contains useful details like which element was clicked or exactly where the mouse was positioned.

Question 2: How does the `addEventListener()` method work in JavaScript? Provide an example.

The `addEventListener()` method is a way to tell the browser to "listen" for a specific event—like a click, keypress, or mouse movement—on a particular element. When that event occurs, the browser executes a pre-defined block of code called a callback function.

How it Works:

1. **Select the Target:** You first identify the DOM element you want to watch (e.g., a button or a div).
2. **Attach the Listener:** You call `addEventListener()` on that element.
3. **Specify Parameters:** You provide at least two required pieces of information:
4. **Event Type:** A string representing the event (e.g., "click", "mouseover").
5. **Callback Function:** The code that runs when the event is triggered.
6. **Wait for Trigger:** The browser continuously monitors the element. Once the specified event happens, the callback function is fired automatically.

Basic Syntax:

```
element.addEventListener(event, function, useCapture);
```

- **event:** (Required) Name of the event. Note: Do not use the "on" prefix (use "click", not "onclick").
- **function:** (Required) The function to execute.
- **useCapture:** (Optional) A boolean (default false) that controls whether the event should be handled in the capturing or bubbling phase.

Code Example: Button Click

This example changes the text of a paragraph when a button is clicked.

HTML:

```
<button id="myBtn">Click Me!</button>
<p id="message">Waiting for click...</p>
```

JavaScript:

```
// 1. Select the elements
const button = document.getElementById("myBtn");
const message = document.getElementById("message");

// 2. Define the callback function
function updateText() {
    message.innerText = "Button was clicked!";
}

// 3. Attach the event listener
button.addEventListener("click", updateText);
```

DOM Manipulation

Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

The Document Object Model (DOM) is a programming interface that represents a web page as a structured tree of objects. It acts as a "bridge" or "translator," allowing JavaScript to understand and manipulate the HTML document that the browser has loaded into memory.

When a browser loads a web page, it parses the HTML and creates a DOM Tree.

- **Nodes:** Every part of the page—the document itself, tags (elements), text inside tags, and attributes (like id or class)—is represented as a "node" in this tree.
- **Hierarchy:** The structure is hierarchical, with the document object as the root. For example, the `<html>` tag is a child of the document, and it contains children like `<head>` and `<body>`.

How JavaScript Interacts with the DOM

JavaScript uses the DOM API—a collection of built-in properties and methods—to interact with the page. This interaction generally happens in four ways:

1. **Selecting Elements:** Finding specific parts of the page to work with.
 - `document.getElementById("id")`: Selects a single unique element.
 - `document.querySelector(".class")`: Selects the first element matching a CSS selector.
 - `document.querySelectorAll("p")`: Selects all matching elements as a list.
2. **Modifying Content & Styles:** Changing what is visible on the screen.
 - `element.textContent = "Hello"`: Changes the text within an element.
 - `element.innerHTML = "<b>Bold</b>"`: Changes the HTML content.
 - `element.style.color = "red"`: Dynamically updates CSS properties.
3. **Creating & Deleting Elements:** Altering the page structure.
 - `document.createElement("div")`: Generates a new element in memory.
 - `parent.appendChild(newElement)`: Adds the new element to the page.
 - `element.remove()`: Deletes an element from the document.
4. **Listening for Events:** Reacting to user actions.
 - `element.addEventListener("click", function)`: Triggers a function when a user clicks a button, submits a form, or hovers over an image.

Question 2: Explain the methods `getElementById()`, `getElementsByClassName()`, and `querySelector()` used to select elements from the DOM.

Selecting elements from the Document Object Model (DOM) is a core task in web development for manipulating page content, styles, and behavior. These three methods offer different ways to target elements based on their unique attributes or CSS patterns.

1. `getElementById()`

This method finds and returns a single element that has a specific id attribute.

- **Behavior:** It returns the first element it finds with the matching ID. Since IDs are intended to be unique within a document, this should ideally target one specific element.

- **Return Value:** An Element object if found, or null if no match exists.
- **Performance:** Generally the fastest method because browsers often use an internal map to track IDs.
- **Syntax:** `document.getElementById("my-unique-id")`.

2. `getElementsByClassName()`

This method searches for all elements that share a specific class name.

- **Behavior:** It returns a collection of all matching descendant elements. It can take multiple class names separated by spaces (e.g., "class1 class2") to find elements that have all specified classes.
- **Return Value:** A live HTMLCollection. "Live" means if the DOM changes (e.g., a new element with that class is added later), the collection updates automatically.
- **Scope:** Unlike `getElementById`, this can be called on a specific element to search only within its children.
- **Syntax:** `document.getElementsByClassName("my-class-name")`.

3. `querySelector()`

This is a more modern and flexible method that uses CSS selectors to find elements.

- **Behavior:** It returns the first element that matches the specified CSS selector(s). You can use complex selectors like tags, classes, IDs, attributes, or nested relationships (e.g., "div > p.active").
- **Return Value:** The first Element object found, or null.
- **Versatility:** It is highly versatile because it can replace almost any other selector method by simply changing the CSS string.
- **Syntax:** `document.querySelector(".my-class")` or `document.querySelector("#my-id")`.

Feature	<code>getElementById</code>	<code>getElementsByClassName</code>	<code>querySelector</code>
Selects By	ID only	Class(es)	Any CSS Selector
Returns	Single Element or null	Live HTMLCollection	First matching Element
Performance	Fastest	Fast	Slightly slower
Use Case	Unique elements (headers, forms)	Groups of elements	Complex or specific CSS patterns

JavaScript Timing Events (setTimeout, setInterval)

Question 1: Explain the setTimeout() and setInterval() functions in JavaScript. How are they used for timing events?

In JavaScript, setTimeout() and setInterval() are native functions used to schedule code execution at specific times. These are asynchronous methods, meaning they don't block the rest of your script while waiting for the timer to expire.

1. setTimeout()

The setTimeout() method executes a function once after a specified delay in milliseconds (1 second = 1000 ms).

Syntax: let timerId = setTimeout(function, delay, arg1, arg2, ...);

How it works: When called, it starts a timer in the background. Once the time is up, the function is added to the JavaScript Event Loop's task queue to be executed.

Clearing: You can stop the execution before it happens by calling clearTimeout(timerId).

Example:

```
const greeting = setTimeout(() => {  
  console.log("Hello after 2 seconds!");  
}, 2000);
```

2. setInterval()

The setInterval() method executes a function repeatedly, with a fixed time delay between each call.

Syntax: let intervalId = setInterval(function, delay, arg1, arg2, ...);

How it works: It continuously schedules the function at regular intervals until it is explicitly stopped.

Clearing: To stop the repetition, you must call clearInterval(intervalId).

Example:

```
let count = 0;  
const counter = setInterval(() => {  
  count++;  
  console.log(`Interval count: ${count}`);  
  if (count === 5) clearInterval(counter); // Stop after 5 times  
}, 1000);
```

Feature	setTimeout()	setInterval()
Execution	Runs once.	Runs repeatedly.
Primary Use	Delays and one-time events (e.g., alert popups).	Periodic updates (e.g., clocks, polling).
Stopping	clearTimeout().	clearInterval().
Precision	Guaranteed delay between end of code and next run if nested.	May skip or overlap if the function takes longer than the interval.

Question 2: Provide an example of how to use setTimeout() to delay an action by 2 seconds.

To delay an action by 2 seconds in JavaScript, you use the setTimeout() function with a delay of 2000 milliseconds.

Basic Example:

This example uses an arrow function to log a message to the console after the 2-second delay:

```
// Delaying an action by 2 seconds (2000ms)
setTimeout(() => {
  console.log("This message appeared after 2 seconds!");
}, 2000);
```

Key Components:

Callback Function: The first argument is the code you want to run (e.g., () => { ... }).

Delay: The second argument is the time to wait in milliseconds (1 second = 1000ms).

Asynchronous Nature: Code written after the setTimeout block will run immediately without waiting for the timer to finish.

Using an Existing Function:

You can also pass a reference to a pre-defined function:

```
function greet() {
  console.log("Hello after 2 seconds!");
}

// Pass the function name, NOT greet() with parentheses
setTimeout(greet, 2000);
```

JavaScript Error Handling

Question 1: What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.

Error handling in JavaScript is the process of managing and responding to runtime errors (also called exceptions) to prevent a program from crashing unexpectedly. By using structured blocks, you can "catch" these errors and execute alternative code to keep the application running smoothly.

The try...catch...finally Blocks

The primary mechanism for error handling in JavaScript consists of three main parts:

- **try:** Wraps the code that might potentially throw an error. JavaScript executes this block first.
- **catch:** Executes only if an error occurs within the try block. It receives an Error object containing details like the error's name and message.
- **finally:** Executes unconditionally after the try and catch blocks finish, regardless of whether an error was thrown or handled. It is commonly used for cleanup tasks like closing files or stopping loading spinners.

Code Example:

This example demonstrates a safe division function that handles potential errors and ensures cleanup.

```
function safeDivision(numerator, denominator) {  
  try {  
    console.log("Starting division...");  
    if (denominator === 0) {  
      // Manually trigger an error if dividing by zero  
      throw new Error("Cannot divide by zero!");  
    }  
    let result = numerator / denominator;  
    console.log("Result:", result);  
  } catch (error) {  
    // This block runs only if an error occurred in the try block  
    console.error("Error caught:", error.message);  
  } finally {  
    // This block ALWAYS runs, successful or not  
    console.log("Process complete. Cleaning up resources...");  
  }  
}  
  
// Case 1: Successful execution  
safeDivision(10, 2);  
  
// Case 2: Error execution
```

```
safeDivision(10, 0);
```

Question 2: Why is error handling important in JavaScript applications?

Error handling is critical in JavaScript because a single unhandled exception can stop the execution of your entire script, leading to unexpected application crashes. Since JavaScript is loosely typed and relies heavily on asynchronous operations, it is prone to runtime issues like invalid user input, network failures, or missing API data.

Key Reasons for Importance

Application Stability: It ensures your program continues to run or fails gracefully instead of crashing and leaving users with a broken interface or a blank page.

User Experience (UX): Instead of silent failures that leave users confused, good error handling provides meaningful feedback or fallback UIs (like "Please check your internet connection") that guide users on what to do next.

Easier Debugging: Structured error handling captures diagnostic information like stack traces and error messages, which helps developers pinpoint and fix bugs faster.

Security: Properly managed errors prevent sensitive information (like database credentials or detailed system paths) from being leaked to the end user in raw error messages.

Data Integrity: It helps prevent data corruption by stopping invalid operations before they can save incorrect data to a database or local storage.